IE415: Control of Autonomous Systems
Visual–Inertial SLAM with Fiducial Landmark Anchors

Semester Project Handout

Project Brief

Develop a complete indoor mapping and localization system using visual and inertial sensing. The system shall estimate the motion of a person carrying the sensing platform, construct or use a map of fixed ArUco markers, and display the estimated trajectory with respect to the reconstructed room map.
The project may be implemented as a mobile application, desktop application, phone–laptop hybrid system, offline processing pipeline, or embedded system. Real-time operation is desirable but not mandatory. The emphasis is on building a functioning autonomous-systems pipeline that integrates sensing, estimation, mapping, visualization, and quantitative evaluation.

1 Problem Statement

A person enters an indoor environment containing ArUco markers placed at fixed locations. The sensing platform records camera images and inertial measurements while the person walks through the room. The system shall perform visual–inertial odometry or SLAM, detect the ArUco markers, estimate their poses in a common reference frame, and show the motion of the sensing platform relative to the resulting map.

The ArUco markers shall serve one or more of the following roles:

- **Landmarks:** marker poses are initially unknown and are estimated as part of the map;
- **Anchors:** known marker poses attach the local SLAM estimate to a fixed room coordinate frame;
- **Loop-closure identifiers:** repeated detection of a uniquely identified marker supplies an unambiguous re-observation constraint;
- **Relocalization references:** a previously constructed marker map is used to localize the sensing platform during a later run.

The markers must support the SLAM system rather than replace it. The trajectory should continue to be estimated when no marker is visible, using visual odometry, inertial propagation, or both.

2 Learning Objectives

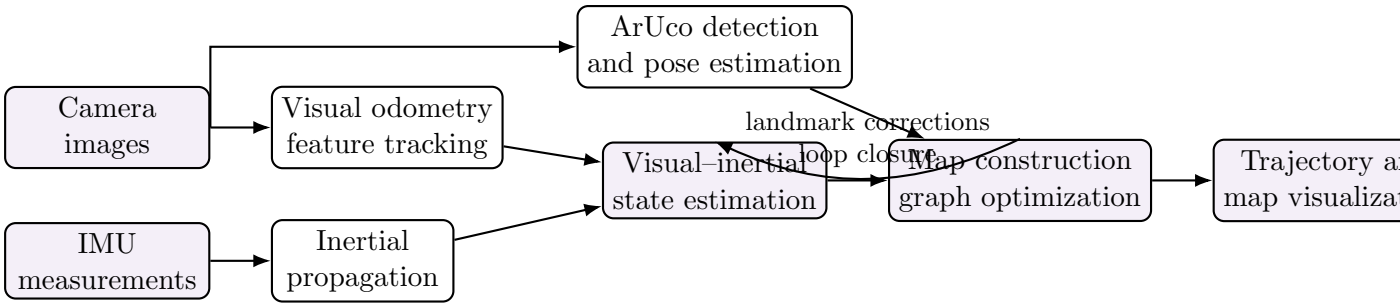
On successful completion of the project, students should be able to:

1. formulate an indoor localization and mapping problem using coordinate frames and rigid-body transformations;
2. acquire and synchronize camera and inertial measurements;
3. calibrate the camera and, where applicable, the camera–IMU transformation;
4. estimate platform motion using visual, inertial, or visual–inertial methods;

5. represent ArUco detections as landmark measurements;
6. construct a persistent landmark map and estimate the platform trajectory in a common frame;
7. use repeated landmark observations for correction, relocalization, or loop closure;
8. design an interactive system that allows a user to capture, process, inspect, and visualize data;
9. evaluate localization and mapping accuracy through repeatable experiments.

3 System Concept

A typical processing pipeline is shown below.



The workflow may be online, offline, or hybrid. Students are encouraged to separate data acquisition from estimation so that the same dataset can be replayed while algorithms are improved.

4 Suggested Implementation Architectures

All architectures below are acceptable and carry the same maximum marks. The platform itself is not the main grading criterion; the quality of the complete system is.

4.1 Architecture A: Mobile System

A smartphone or tablet provides camera and IMU measurements. Processing may run fully on the device or partly on another computer.

- Android or iOS application;
- live image and sensor acquisition;
- start/stop recording and dataset management;
- on-device or remote SLAM processing;
- mobile visualization of the map and trajectory.

4.2 Architecture B: Desktop System

A laptop or desktop performs both acquisition and processing.

- built-in or USB webcam;
- external IMU connected through Arduino, ESP32, or another interface;

- Python, C++, MATLAB, ROS2, or another suitable framework;
- desktop-based processing and visualization.

4.3 Architecture C: Phone–Laptop Hybrid

A phone acts as the sensing and user-interaction device, while a laptop performs computationally intensive processing.

- phone records video, IMU measurements, and timestamps;
- data is transferred by cable, Wi-Fi, Bluetooth, or file upload;
- the laptop performs visual–inertial SLAM and map optimization;
- results are displayed on the laptop, returned to the phone, or shown through a web interface.

4.4 Architecture D: Embedded System

A suitable embedded board integrates the camera, IMU, computation, storage, and optionally communication.

Representative platforms include Raspberry Pi, NVIDIA Jetson, BeagleBone, a sufficiently capable microprocessor board, or a heterogeneous system combining a microcontroller with a companion computer.

Possible implementations include:

- Raspberry Pi or Jetson with CSI/USB camera and IMU;
- embedded Linux board running OpenCV, ROS2, or a SLAM library;
- microcontroller for synchronized IMU acquisition and a companion processor for vision;
- fully self-contained handheld mapping device;
- camera–IMU data logger followed by offline processing.

Students selecting this architecture should explicitly discuss computational, memory, power, bandwidth, and timing constraints.

4.5 Architecture E: Offline or Post-hoc Processing

Students may record a dataset first and process it later. Real-time operation is not compulsory.

A valid offline pipeline may consist of:

1. recording RGB video, IMU measurements, and timestamps;
2. synchronizing and calibrating the recorded streams;
3. replaying the data through the estimation pipeline;
4. reconstructing the trajectory and marker map;
5. evaluating different algorithmic choices on the same dataset.

5 Mathematical Formulation

A visual–inertial state may be represented as

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{p}_k & \mathbf{v}_k & \mathbf{q}_k & \mathbf{b}_{a,k} & \mathbf{b}_{\omega,k} \end{bmatrix}^T,$$

where $\mathbf{p}_k$ is position, $\mathbf{v}_k$ is velocity, $\mathbf{q}_k$ represents orientation, and $\mathbf{b}_{a,k}$, $\mathbf{b}_{\omega,k}$ are accelerometer and gyroscope biases.

Suppose marker M_i is detected in camera frame C_k. Marker detection provides a relative transformation

$$^{C_k}T_{M_i}.$$

If the global marker pose $^{W}T_{M_i}$ is known, the camera pose can be constrained by

$$^{W}T_{C_k} = {}^{W}T_{M_i}\left({}^{C_k}T_{M_i}\right)^{-1}.$$

If the camera pose is already estimated and a previously unseen marker is detected, its initial global pose can be estimated as

$$^{W}T_{M_i} = {}^{W}T_{C_k}\,{}^{C_k}T_{M_i}.$$

A graph-based implementation may jointly optimize platform poses X_k and marker poses L_i:

$$\min_{\{X_k,L_i\}} \sum_k \|\mathbf{r}_{\text{VIO}}(X_k, X_{k+1})\|^2_{\Sigma_k} + \sum_{(k,i)\in\mathcal{O}} \|\mathbf{r}_{\text{marker}}(X_k, L_i, z_{ki})\|^2_{\Lambda_{ki}}.$$

Students are not required to implement a full visual–inertial estimator from first principles. However, they must understand, document, and experimentally analyse the estimator or library used.

6 Minimum Functional Requirements

Every project must demonstrate all of the following:

1. acquisition of a moving camera sequence;
2. acquisition of IMU measurements, unless a technically justified visual-only baseline is approved;
3. timestamping and synchronization of sensor streams;
4. camera calibration;
5. detection and unique identification of ArUco markers;
6. estimation of marker pose relative to the camera;
7. estimation of a continuous platform trajectory;
8. construction or loading of a marker map;
9. display of the platform trajectory relative to the marker map;
10. repeated marker observation used for correction, consistency checking, relocalization, or loop closure;
11. comparison against at least one baseline;
12. quantitative experimental evaluation.

7 Expected User-Facing Build

The project should provide a usable interface rather than only a collection of disconnected scripts. Depending on the selected architecture, the interface may be a mobile application, desktop application, web application, or embedded-device interface.

At minimum, the user should be able to:

- start and stop data acquisition;
- inspect camera and sensor status;
- save or load a dataset;
- initiate processing;
- view detected markers;
- view the estimated trajectory and map;
- inspect basic performance or diagnostic information.

The interface need not be commercially polished, but it must make the system coherent, demonstrable, and usable.

8 Experimental Requirements

The room should contain approximately six to ten markers distributed across walls or fixed objects. Marker size, placement, height, and orientation should be documented.

Each team shall perform at least the following experiments:

1. **Simple path:** a straight, L-shaped, or rectangular trajectory;
2. **Marker-loss path:** a segment in which no marker is visible for a measurable duration;
3. **Closed-loop path:** return to the starting region and re-observe one or more earlier markers;
4. **Repeat run:** use a previously generated map to relocalize in the same room;
5. **Difficult condition:** poor lighting, rapid motion, oblique viewing angle, partial occlusion, sparse markers, or another justified challenge.

Where feasible, students should compare:

visual or visual–inertial estimation only versus estimation with ArUco landmark constraints.

9 Evaluation Metrics

Suitable metrics include:

- translational trajectory error;
- heading or orientation error;
- endpoint drift;
- drift per metre travelled;
- loop-closure residual;
- marker-position and marker-orientation error;

- relocalization error across separate runs;
- time required for relocalization;
- marker detection success rate;
- processing rate or runtime;
- memory and power use for embedded implementations;
- failure rate under difficult conditions.

A basic position error measure is

$$e_p(k) = \|\mathbf{p}_{\text{est}}(k) - \mathbf{p}_{\text{gt}}(k)\|_2 .$$

Ground truth may be obtained through manual room measurements, surveyed marker locations, a measured floor path, motion-capture data where available, or another clearly justified reference method.

10 Suggested Development Milestones

Milestone	Expected outcome
M1	Problem formulation, selected architecture, hardware and software stack, initial system diagram.
M2	Camera acquisition, IMU acquisition, timestamp logging, and camera calibration.
M3	Reliable ArUco detection and camera-to-marker pose estimation.
M4	Visual odometry or visual–inertial trajectory estimation without marker correction.
M5	Marker-map construction, repeated landmark observation, and trajectory visualization.
M6	Drift correction, relocalization, or loop closure; comparison with the baseline.
M7	Final experiments, quantitative evaluation, interface refinement, report, and demonstration.

11 Deliverables

Each team shall submit:

1. **Proposal:** problem interpretation, selected architecture, hardware, software, responsibilities, and milestones;
2. **System design document:** architecture, coordinate frames, data flow, calibration procedure, and estimation method;
3. **Working prototype:** mobile, desktop, hybrid, embedded, or offline system with a usable interface;
4. **Source code and build instructions:** sufficient for reproduction;
5. **Dataset:** at least one representative recorded experiment with metadata;

6. **Technical report:** mathematical formulation, implementation, experiments, results, limitations, and conclusions;
7. **Demonstration video:** concise recording of the complete workflow;
8. **Final presentation and live demonstration.**

12 Assessment Rubric

Component	Weight	Evidence expected
Problem formulation and system architecture	10%	Clear decomposition, justified architecture, appropriate coordinate frames and data flow.
Sensor acquisition, synchronization, and calibration	15%	Reliable data collection, timestamp handling, camera calibration, and camera–IMU alignment where relevant.
State-estimation and SLAM methodology	20%	Correct use and understanding of visual, inertial, filtering, optimization, or SLAM methods.
Marker mapping, anchoring, and relocalization	15%	Proper landmark representation, repeated observations, drift correction, loop closure, or persistent-map use.
System integration and user-facing build	15%	Coherent application or device; capture, processing, visualization, and diagnostics work together.
Experimental design and quantitative evaluation	15%	Repeatable tests, meaningful baseline, appropriate metrics, and critical analysis.
Technical report and presentation	10%	Clear documentation, mathematical explanation, results, limitations, and effective demonstration.
Total	**100%**	

13 Permitted Use of Existing Libraries

Students may use existing libraries or open-source implementations for visual odometry, VIO, SLAM, optimization, and visualization. Examples include OpenCV, ORB-SLAM3, OpenVINS, VINS-Mono, GTSAM, ROS2 packages, ARCore, or ARKit.

However:

- the complete submission must not be a trivial wrapper around a library demo;
- students must clearly identify which components were implemented, modified, configured, or reused;
- the role of ArUco landmarks must be integrated into the estimation or mapping pipeline;
- students must explain the state representation, measurements, transformations, and sources of error;
- evaluation must demonstrate understanding beyond successful execution of third-party code.

Using ARCore or ARKit solely to obtain a finished trajectory and then plotting markers on top will not, by itself, constitute a complete SLAM project.

14 Possible Extensions

Extensions may include:

- optimal marker placement;
- sparse versus dense marker configurations;
- factor-graph optimization;
- EKF-based camera–IMU–marker fusion;
- stereo or depth sensing;
- multi-room or multi-floor mapping;
- embedded real-time optimization;
- multi-user or collaborative mapping;
- semantic objects in addition to markers;
- uncertainty visualization;
- robustness to false detections and marker occlusion;
- automatic map saving and later relocalization;
- comparison of mobile, desktop, and embedded implementations.

15 Final Success Criterion

A successful project should allow an observer to see a complete engineering system:

> A person starts the system, enters or walks through a marked room, records visual and inertial data, and obtains a map showing fixed ArUco landmarks together with the estimated trajectory of the sensing platform. Repeated marker observations improve consistency, and the system reports quantitative evidence of its accuracy and limitations.

The strongest submissions will not merely produce an attractive demonstration. They will show that the students understand why the estimator works, where it fails, how the markers alter observability and drift, and how the complete system can be improved.